

Session 6 Homework — One More Extraction, One New Endpoint, One Frontend Connection

CORE, STRETCH, and CHALLENGE are all required — every one of them must be completed. The three names describe increasing difficulty only, nothing about which parts you submit.

Context

This extends the same B4F Hub project. It is **not** a separate project — it continues from where Session 6 ended: `server/index.js` now mounts two routers (`/api/opportunities`, `/api/posts`), each backed by its own `controllers/*.js` file, with a shared `server/store.js` owning the real, in-memory arrays and `server/utils.js` owning the shared `delay()` helper. Only one route is still inline in `index.js`: `GET /api/health`. This homework finishes that extraction, adds one genuinely new endpoint using the same real validation shape taught this session, and connects one small piece of frontend UI to it.

Goal

Extract the last remaining inline route into its own router/controller pair, add a small new POST endpoint with real backend validation, and wire one frontend action to it.

What to build

A `server/controllers/health.js` + `server/routes/health.js` pair (CORE), a new `POST /api/opportunities/:id/report` endpoint on the already-extracted opportunities router/controller (STRETCH), and a small "report an issue" input + button on the opportunity detail page, calling that new endpoint (CHALLENGE).

CORE (required) — extract GET /api/health

1. Create `server/controllers/health.js`, exporting a `getHealth(req, res)` function that reads `opportunities / posts` from `server/store.js` — the exact logic currently inline in `server/index.js`, moved, not rewritten.
2. Create `server/routes/health.js`: an `express.Router()` with one route, `router.get("/", getHealth)`, exported as the default export — the same shape as `routes/opportunities.js` and `routes/posts.js`.
3. In `server/index.js`, mount it with `app.use("/api/health", healthRouter)` and remove the old inline route. `index.js` should now contain no direct reference to `opportunities` or `posts` at all — every read of either array happens inside a controller.

STRETCH (required) — POST /api/opportunities/:id/report

1. In `server/controllers/opportunities.js`, add a small in-memory `reports` array and a new `reportOpportunity(req, res)` function:
 - read `:id` from `req.params` and `reason` from `req.body` (defensively, `req.body ?? {}`);

- if no opportunity exists with that id, respond `404` ;
 - otherwise, if `reason` is not a string of at least 3 characters (after trimming), respond `400` with a clear error message;
 - otherwise, create a report object (`{ id, opportunityId, reason, createdAt }`), add it to the `reports` array, and respond `201` with the created report.
2. In `server/routes/opportunities.js` , wire it in: `router.post("/:id/report", reportOpportunity)` .

Check the existence of the opportunity **before** validating `reason` — if the id itself doesn't exist, whether the reason is valid is irrelevant, and a `404` is the more useful answer.

Test this directly with curl (or Postman/Thunder Client) before touching the frontend: `curl -X POST http://localhost:3001/api/opportunities/101/report -H "Content-Type: application/json" -d '{"reason": "Listing looks outdated"}'` should return `201` and the new report.

CHALLENGE (required) — connect the frontend

1. In `client/src/api.ts` , add a small `OpportunityReport` type and a `reportOpportunity(id, reason)` function calling the new endpoint, following the exact same `fetch` + `response.ok` + JSON-cast shape every other function in that file already uses.
2. In `client/src/pages/OpportunityDetailPage.tsx` , add a small text input and a "Report" button beneath the existing Apply button, following the same `useState` /`async-handler`/ `useNotify()` pattern already used for `handleApply` . The button should be disabled while the input is empty or a request is in flight.
3. Give the new input and button a small amount of CSS so they look like they belong (not raw, unstyled browser defaults).

Rules

Allowed — everything taught by the end of Session 6: `express.Router()` , `app.use(mountPath, router)` , a controller file exporting plain `(req, res)` functions, `req.body ?? {}` , `res.status(...).json(...)` , the existing `store.js` / `utils.js` sharing pattern, and plain `useState` /`async-handler` frontend wiring (the same shape `handleApply` already uses).

Not yet (do not use these) — a database of any kind, `req.query` , any new npm package, a validation library (Zod, Joi, express-validator), a custom Express error-handling middleware (it was only discussed conceptually this session, never implemented — do not add one here), Redux for this feature, authentication, or CORS configuration. If you find yourself reaching for one of these, you have left the scope of this homework.

Do not change the behaviour of any existing endpoint — this homework only extracts one more route and adds one new one; every route that already worked must keep working identically.

Test it

- `curl http://localhost:3001/api/health` still returns the real counts, exactly as before.

- `curl -X POST http://localhost:3001/api/opportunities/101/report -H "Content-Type: application/json" -d '{"reason": "Listing looks outdated"}'` returns `201` and the created report.
- The same request with `{"reason": "x"}` returns `400`.
- The same request against a nonexistent id (e.g. `99999`) returns `404`.
- With both servers running, open a real opportunity detail page, type a reason, and click Report — a success message appears, the input clears, and the server's own terminal logs the real request.
- `npm run build` and `npm run lint` (in `client/`) both pass with 0 errors. `node --check index.js` (in `server/`) reports no syntax errors.

Done when

- CORE, STRETCH and CHALLENGE are all implemented — all three are required.
- `server/index.js` mounts three routers (`/api/health`, `/api/posts`, `/api/opportunities`) and contains no direct reference to `opportunities / posts` anywhere.
- `POST /api/opportunities/:id/report` correctly returns `404 / 400 / 201` for the three cases above.
- The detail page's Report button works end to end and is visually consistent with the rest of the page.
- `npm run build` and `npm run lint` pass with 0 errors; `node --check index.js` passes.

Working time

Expect about **2 to 3 hours** — CORE is a short, mechanical repeat of a pattern used twice already in class; STRETCH's real work is getting the validation order right (existence before content), not the route itself; CHALLENGE reuses `handleApply`'s own shape almost exactly.

Submitting

Commit and push to your own `student-<your-github-username>` branch, as usual. Do not push to `main`.